VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Chosen Software: **Git — Version Control System (GPL v2)**

Student Name	Registration Number
PRAGATI KUMSRI	24BCE10433
Slot	**Date of Submission**
D11	30-03-2026

Introduction & Software Choice

Open source software is everywhere, even if most people don't realise it. The servers behind your favourite websites, the tools developers lean on every single day, the kernel running inside your Android phone — almost none of that was written by one company and sold in a box. It was written openly, shared freely, and improved collectively over decades. That's the spirit this project asks you to sit with for a moment.

For this audit, I've chosen Git — the distributed version control system originally written by Linus Torvalds in 2005. It's released under the GNU General Public License version 2 (GPL v2), and it's one of the most consequential pieces of software ever written. Not because it's glamorous, but because without it, modern software development as we know it simply wouldn't function. Every pull request, every CI/CD pipeline, every collaborative codebase on GitHub or GitLab sits on top of what Linus built in a few weeks out of frustration.

What makes Git especially interesting to audit isn't just the technical brilliance — it's the story behind it. Git was born out of a proprietary relationship gone wrong. The Linux kernel team was using a tool called BitKeeper, and when that arrangement collapsed, Linus didn't wait for someone else to solve the problem. He built something better himself, and then gave it away. That decision to share, rather than control, ended up reshaping the entire industry.

Why this matters

Git isn't just a tool — it's a philosophy in code. Understanding why it exists, and who it was built for, is the whole point of this audit.

Chosen Software at a Glance

Software	Category	License	Why It's Interesting
Git	Version Control	GPL v2	The tool Linus built when proprietary failed him

Part A1 — Origin & Philosophy: The Problem

What Version Control Looked Like Before 2005

To understand why Git exists, you have to go back to a time when version control was, frankly, a mess. The dominant tools of the early 2000s were CVS — the Concurrent Versions System — and its successor SVN (Subversion). Both are centralised systems, which means there's one server somewhere that holds the 'true' history of the project, and everyone else checks out and checks in from that single source of truth.

In my experience reading through the history, the frustrations people had with CVS were almost comical in hindsight. Renaming a file was a nightmare. Merging branches was slow and painful. If the central server went down, nobody could work — full stop. SVN improved things somewhat, but it was still fundamentally centralised. And for a project like the Linux kernel, which had hundreds of contributors spread across the globe submitting patches simultaneously, centralisation was a genuine bottleneck.

The Linux Kernel's Reliance on BitKeeper

Here's where it gets interesting. Between 2002 and 2005, the Linux kernel development team was actually using a proprietary version control tool called BitKeeper, built by a company called BitMover. Larry McVoy, BitKeeper's creator, allowed the kernel developers to use it for free — but that arrangement came with strings attached. It was a commercial tool being used gratis, and everyone in the free software community knew that dependency was a problem waiting to happen.

And then it happened. In April 2005, the free-use licence was revoked. The exact circumstances are somewhat disputed — it involved a kernel developer attempting to reverse-engineer BitKeeper's protocols — but the outcome was unambiguous: the Linux kernel suddenly had no version control system. For a project of that scale and velocity, that was a crisis.

What Was Broken About the Alternatives

Linus looked at the available free alternatives — CVS, SVN, Arch, Monotone — and found none of them acceptable. Not because they were bad tools for ordinary projects, but because the Linux kernel has requirements that most software projects simply don't. Consider the numbers: thousands of patches per release cycle, contributions from hundreds of developers simultaneously, a need for cryptographic integrity checking so nobody could tamper with the history, and speed — real speed, not 'fast enough' speed.

CVS had no atomic commits — if your commit crashed halfway through, you'd have a half-applied change in the repository with no clean way to roll back. SVN fixed that, but introduced its own branch-handling quirks and remained painfully slow for large trees. Arch was theoretically distributed but had a reputation for being arcane and slow. None of them handled merging at the scale the kernel needed.

The core problem

There was no free, distributed, fast version control system that could handle a project the size and complexity of the Linux kernel. That gap was the entire reason Git came to exist.

Part A1 — Origin & Philosophy: The Solution

Why Linus Decided to Build It Himself

Linus Torvalds is not, by his own admission, someone who typically writes tools. He writes operating system kernels. But in April 2005, after the BitKeeper situation collapsed, he spent about two weeks building Git from scratch — not because he wanted to, but because nothing else met his requirements. He later joked that his naming convention was to name things after himself: first Linux, then Git. (The word 'git' is British slang for an unpleasant person, which he thought was fitting.)

What I find genuinely remarkable is the speed. By June 2005 — less than three months after starting — Git was already managing the Linux kernel's own source code. That's not just fast development; that's a statement about the clarity of the design. Linus knew exactly what he wanted to build before he wrote the first line.

The Core Design Philosophies

Git's architecture is built around three non-negotiable principles that Linus laid out from the beginning. Understanding these isn't just trivia — they explain why Git behaves the way it does and why it won out over everything else.

1. Distributed by design

In Git, every developer has a complete copy of the entire repository history on their machine. There is no single server you must connect to in order to commit, branch, or view history. You can work entirely offline and push your changes later. This wasn't just a convenience feature — it was a philosophical position about how software collaboration should work. No single point of failure, no gatekeeping server, no dependency on anyone else's infrastructure.

2. Speed as a first-class requirement

Linus explicitly said he wanted operations to be fast — not 'reasonably fast' but 'laughably fast'. Branching in Git is nearly instantaneous because a branch is just a pointer to a commit object. Merging is fast because Git tracks content rather than files. Even on a repository the size of the Linux kernel — which by now has millions of commits — most local operations complete in milliseconds.

3. Cryptographic integrity

Every object in Git — every file, every directory tree, every commit — is identified by a SHA-1 hash of its contents. This means the history is tamper-evident by construction. If someone modifies a commit from three years ago, its hash changes, and every subsequent commit that referenced it also becomes invalid. For the kernel, where the integrity of the codebase is a security matter, this was essential.

Origin■s tory	Git was built in April 2005 by Linus Torvalds in response to the collapse of the kernel's BitKeeper licence. It took roughly two weeks to write and was managing the Linux kernel itself by June 2005.

Part A2 — The License: GPL v2 & The Four Freedoms

Git is distributed under the GNU General Public License version 2, the same licence that covers the Linux kernel. This isn't a coincidence — Linus chose it deliberately, and understanding what GPL v2 actually says is essential to understanding why Git has spread the way it has.

The Four Freedoms of Free Software

The Free Software Foundation defines free software through four fundamental freedoms. A program is only truly free if it grants all four:

Freedom 0: The freedom to run the program for any purpose, without restriction.

Freedom 1: The freedom to study how the program works and adapt it to your needs. Access to the source code is a prerequisite.

Freedom 2: The freedom to redistribute copies so you can help others.

Freedom 3: The freedom to distribute copies of your modified versions to others. This allows the whole community to benefit from your improvements.

Does Git's GPL v2 Grant All Four?

Yes — unambiguously. Git's source code is publicly available at kernel.org and on GitHub. You can download it, read every line, compile it yourself, modify it, and redistribute your version. Freedom 0 is satisfied because Git can be used for any purpose, whether you're managing personal dotfiles or coordinating a team of a thousand developers. Freedoms 1, 2, and 3 are satisfied because the full source is always available and redistribution — even of modified versions — is explicitly permitted.

The Copyleft Clause: Can You Modify and Sell Without Sharing?

This is where GPL v2 has teeth. The licence contains what's called a copyleft clause: if you distribute a modified version of Git, you must also distribute the source code of your modifications under GPL v2. You cannot take Git, make proprietary improvements, and ship a closed binary. The moment you distribute a derivative, the four freedoms must pass to your recipients too.

Importantly, the restriction only kicks in when you distribute. A company can absolutely modify Git internally — adding custom hooks, altering behaviour for their pipeline — and keep those changes private, as long as they never distribute the modified binary outside the organisation. This is the SaaS loophole that GPL v2 doesn't close (though GPL v3 and AGPL attempt to).

Part A2 — License Comparisons & Concepts

GPL vs MIT: A Fundamental Difference in Philosophy

The GPL and the MIT licence represent two very different answers to the question of what 'open source' should mean. MIT is permissive: you can take the code, do almost anything with it — including incorporating it into closed-source proprietary software — as long as you retain the copyright notice. The freedom runs only one direction: from the author to the user. Once it's out in the world, MIT code can be absorbed and locked up.

GPL is different. It's designed to ensure that freedom is self-perpetuating. If you distribute GPL code (or a derivative of it), the recipients must receive the same freedoms you did. It's sometimes described as 'viral' — and while that's often said pejoratively, it's actually the whole point. Richard Stallman designed it this way deliberately, to prevent the commons from being privatised.

Dimension	GPL v2	MIT
Can modify?	Yes	Yes
Can distribute?	Yes, with source	Yes, freely
Can make proprietary?	No (on distribution)	Yes
Used by Git/Linux	Yes	No
Used by React, Rails, Node	No	Yes

Which Would I Choose for My Own Projects?

Honestly? It depends on what I'm trying to accomplish. If I'm building something that I want the community to benefit from — and I want to ensure that benefit isn't privatised — I'd choose GPL. But if I'm writing a library and I want maximum adoption, including by companies who would otherwise avoid GPL code in their proprietary products, MIT makes more practical sense. There's no universally correct answer; it's a values question masquerading as a legal question.

Free as in Beer vs Free as in Freedom

This distinction matters more than it might initially seem. 'Free as in beer' means zero cost — you can download Git for nothing. But that's not the important kind of free. 'Free as in freedom' means you have the right to understand, modify, and share the tool. Proprietary software can also be zero cost — plenty of it is. What makes Git genuinely free in the meaningful sense is that no company can revoke your right to use your own copy, no update can silently change behaviour you depend on, and the knowledge of how it works is yours to have.

Consider the contrast with BitKeeper, the proprietary tool Git replaced. BitKeeper was free-as-in-beer for Linux kernel developers — until it wasn't. The licence was revoked and the whole project was suddenly stranded. That

Part A3 — The Ethics of Open Source

Should All Software Be Open Source? The Case For

There's a compelling argument that software which runs critical infrastructure — hospitals, voting systems, financial markets, telecommunications — should by definition be open to public scrutiny. Security through obscurity is a known failure mode; the only reliable way to audit a system for backdoors, vulnerabilities, or discriminatory algorithms is to examine the source code. When software affects public life, the public arguably has a right to understand it.

Beyond security, there's an innovation argument. Open source creates a commons — a shared pool of tools that anyone can build on without paying rent. The entire modern internet was built on open-source foundations: Apache, Linux, OpenSSL, GCC, Python. Imagine if any of those had been proprietary. The cost of entry for new participants would be radically higher, and the rate of innovation would be correspondingly slower.

The Case Against

That said, the argument doesn't go all the way without some pushback. Open sourcing software doesn't automatically produce a community of contributors. Most open source projects are maintained by one or two people, working evenings and weekends, for no pay. Mandating that all software be open is one thing; ensuring that it's actually maintained and improved is quite another. The Log4Shell vulnerability in 2021 — which affected millions of systems globally — was found in a library maintained by a tiny volunteer team with essentially no resources. Exposure without support can be its own kind of risk.

Is It Ethical for Corporations to Profit from Community Labor?

This is the question I find most interesting — and most uncomfortable. Huge corporations like Google, Meta, Microsoft, and Amazon collectively contribute billions of dollars of economic value built on Linux, Git, and other GPL-licensed tools. Some of them contribute back significantly; many don't. Is that ethical?

I think the honest answer is: it's complicated. GPL v2 doesn't require payment — it requires sharing modifications if you distribute them. Companies that use Git internally but never distribute a modified binary are fully within their rights. Whether they have a moral obligation to contribute is a different question from whether they have a legal one. Red Hat built a multi-billion dollar company on Linux and was also one of the largest contributors. Amazon runs massive infrastructure on open-source tools and contributes selectively. Both positions are legal. Only one, I'd argue, is genuinely ethical.

Standing on the Shoulders of Giants

Every developer writing code today is standing on decades of open-source

Part B — Linux Footprint: Installation & Directories

Installation on WSL (Ubuntu)

Installing Git on a Debian/Ubuntu-based system — including WSL — is straightforward via the apt package manager. The community maintains a PPA that usually ships a more recent version than the Ubuntu default repositories, but for most purposes the default is fine.

```
# Update the package index first
sudo apt update

# Install Git
sudo apt install git -y

# Verify installation
git --version
# Output: git version 2.43.0
```

Key Directories and Files

`/usr/bin/git`
The main Git binary. This is what runs when you type 'git' at the terminal.

`/usr/lib/git-core/`
Contains sub-command binaries (git-merge, git-fetch, etc.) that the main binary delegates to.

`~/.gitconfig`
Per-user configuration file. Stores your name, email, preferred editor, aliases. Created when you run 'git config --global'.

`/etc/gitconfig`
System-wide configuration. Applies to all users on the machine. Typically used by admins in shared environments.

`/usr/share/doc/git/`
Documentation and changelog. Not required for operation, but useful for auditing what's installed.

User and Group — Why It Matters for Security

Git runs under the same user and group as whoever invokes it — there's no special git system user, no privilege escalation. If you run 'git clone' as your regular user account, Git operates with exactly your permissions. This is by design: Git is a local tool that doesn't need elevated access, and keeping it unprivileged limits the blast radius of any vulnerability. You can verify this with:

```
# See which user owns the Git binary
ls -la $(which git)
# -rwxr-xr-x 1 root root ... /usr/bin/git

# Git runs as the invoking user — check with:
git config --list
whoami
```

Part B — Linux Footprint: Management & Updates

Git Is Not a Daemon — How It Operates Differently

Unlike a web server (Apache, Nginx) or a database (MySQL, PostgreSQL), Git is not a persistent background service. It doesn't run continuously, doesn't listen on a port, and doesn't have a process you can 'stop' or 'restart'. Git is command-driven: it starts when you invoke it, does its work, and exits. There is no 'git.service' in systemd because there's nothing to manage between invocations.

The exception is git-daemon — a lightweight server that can serve Git repositories over the git:// protocol. But this is rarely used in modern setups, which tend to rely on SSH or HTTPS instead. If you were running a bare server with git-daemon, then you would manage it as a service:

```
# git-daemon (rarely needed — most use SSH or HTTPS)
sudo systemctl start git-daemon.socket
sudo systemctl enable git-daemon.socket
sudo systemctl status git-daemon.socket

# But for normal Git usage — no service needed.
# Just invoke git directly:
git status
git log --oneline -10
```

The Update Model — Security Patches via apt

Security patches to Git reach your system through the distribution's package manager. When the Git security team (which operates primarily through the kernel.org mailing list and the git@vger.kernel.org list) identifies and patches a vulnerability, they release a new version. Distribution maintainers pick up that version, package it, and push it to the apt repositories. Your machine picks it up on the next 'apt upgrade'.

```
# Check current version
git --version

# Update Git (and all other packages)
sudo apt update && sudo apt upgrade git

# Check for security advisories
apt-get changelog git | head -40

# Or via dpkg — see installed version
dpkg -l git
```

What I find interesting about this model is that it's genuinely community-driven. Nobody is pushing mandatory updates at you on a schedule determined by a corporation's release cadence. You decide when to update, and the update process is transparent — you can read the changelog, inspect the

Security model Git CVEs are published openly. Anyone can read the patch, verify the fix, and audit the change before deploying it. No vendor holds the keys to your update pipeline.

Part C — The FOSS Ecosystem

Dependencies: What Git Relies On

Git doesn't exist in isolation — it sits on top of several critical open-source libraries. Understanding these dependencies is part of understanding the ecosystem:

`zlib`	Compression. Git compresses object data using zlib, which dramatically reduces repository size. Every .pack file is zlib-compressed.
`libcurl`	Network transport. When you push/pull over HTTPS, libcurl handles the actual HTTP connections, including authentication and SSL/TLS.
`OpenSSL / GnuTLS`	Cryptographic operations. Used for HTTPS transport and (on some builds) SSH-related operations.
`POSIX shell`	Many Git commands are thin shell scripts wrapping core plumbing commands. Git's portability depends on POSIX compliance.
`Perl`	Several older Git utilities are written in Perl (git svn, git send-email), though newer code tends to be C or shell.

What Git Has Enabled

The platforms and tools that Git has made possible are genuinely staggering in scope. GitHub, launched in 2008, built its entire product on top of Git and became the largest code hosting platform in the world — acquired by Microsoft in 2018 for $7.5 billion. GitLab followed, offering a self-hosted alternative with a full DevOps suite. Bitbucket, Gitea, Forgejo — all Git-native platforms.

Beyond hosting, Git is the backbone of modern CI/CD. Every GitHub Actions workflow, every Jenkins pipeline, every GitLab CI job starts with a git clone. Container image builds, infrastructure-as-code with Terraform, Kubernetes manifest management with Argo CD — all assume Git as the source of truth. The term 'GitOps' exists precisely because Git became the de facto state machine for infrastructure.

Governance and Community

Git's development happens primarily through the git@vger.kernel.org mailing list — a public, archived list where patches are submitted as emails, reviewed publicly, and applied by the maintainer. Junio C Hamano has been the primary maintainer since Linus handed off the project in July 2005, just a few months after creating it. The governance is loose but effective: no foundation, no formal board, just a mailing list, a trusted maintainer, and a culture of code review.

Part D — Open Source vs Proprietary

Git's closest proprietary competitors are Perforce (now Helix Core) and Microsoft's Team Foundation Version Control (TFVC), both widely used in enterprise settings — particularly in game development and large corporate engineering teams. Here's an honest comparison:

Dimension	Git (Open Source)	Perforce / TFVC (Proprietary)
Cost	Free. No licencing fees, no per-seat costs. Self-hosting is	Perforce: expensive per-user licencing. TFVC: bundled with
Security (code auditability)	Full source available. Anyone can audit for backdoors or	Closed source. You trust the vendor's security claims.
Support & reliability	Community support via mailing list, Stack Overflow, GitHub	Paid support with SLAs. Dedicated enterprise support
Freedom to modify	Full freedom under GPL v2. Fork it, patch it, build on it — as	None. Source is proprietary. You depend entirely on the
Community vs corporate control	Governed by a volunteer maintainer and mailing list	Controlled by Perforce Software / Microsoft. Features driven by
Scalability	Excellent for distributed teams. Struggles with very	Perforce handles massive monorepos and large binaries
Overall verdict	Recommended for almost all software development contexts.	Legitimate choice for specific use cases (binary-heavy, strict

Part D — Verdict & Conclusion

Would I Recommend Git for Real-World Deployment?

Without hesitation, yes. Git is not just 'good enough' — it's the standard against which every other version control system is now measured. It handles distributed teams elegantly, supports complex branching strategies (Gitflow, trunk-based development, whatever you need), integrates with every major CI/CD platform, and has an ecosystem of tooling that no proprietary alternative can match. The learning curve is real — Git's mental model takes time to internalise — but the investment pays off almost immediately.

The only contexts where I'd seriously consider an alternative are those involving very large binary assets — game engines with gigabyte texture libraries, for instance, where Perforce's handling of large files is genuinely superior. But even there, Git LFS has closed the gap considerably. For software development of any kind, Git is the right choice. The fact that it's free, auditable, and governed by a community rather than a vendor is not just a philosophical bonus — it's a concrete risk reduction.

Would I Contribute to Git?

This is the more interesting question. Contributing to Git means working through the mailing list workflow — formatting patches with 'git format-patch', sending them via 'git send-email', and participating in the review culture that Junio Hamano maintains. It's old-school, and deliberately so. The quality bar is high and the review process is rigorous, which is why Git's codebase has remained remarkably stable over two decades.

I think the honest answer is: I'd want to, yes. Working on a codebase that every developer on earth touches daily is a meaningful thing. Even small contributions — documentation improvements, test coverage, minor bug fixes — matter at that scale. The barrier to entry isn't the technical difficulty; it's the mailing list workflow, which feels alien if you've only ever used GitHub pull requests. But that's a solvable problem.

A closing thought

Doing this audit changed how I think about the tools I use every day. Git was built because someone was frustrated and decided to fix the problem — and then gave the fix away. The whole modern software industry is downstream of that decision. The next time I run 'git commit', I'll remember that someone stayed up late in April 2005 and wrote the thing for free. That's what open source actually is.

Shell Script Documentation — Scripts 1 & 2

Script 1 — System Identity Report

This script greets the user with a formatted summary of their Linux environment. It demonstrates basic variable assignment, command substitution using $() to capture the output of system commands like uname -r and whoami, and formatted output with echo. The script also reads /etc/os-release to extract the distribution name, which is more portable than distro-specific commands.

```
#!/bin/bash
# Script 1: System Identity Report
# Author: [Your Name] | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="[Your Name]"
SOFTWARE_CHOICE="Git"

# --- System info via command substitution ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DISTRO=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '"')
CURRENT_DATE=$(date '+%A, %d %B %Y  %H:%M:%S')
HOME_DIR=$HOME

# --- Display formatted output ---
echo "========================================"
echo "  Open Source Audit — $STUDENT_NAME"
echo "  Software: $SOFTWARE_CHOICE (GPL v2)"
echo "========================================"
echo "Distribution : $DISTRO"
echo "Kernel       : $KERNEL"
echo "User         : $USER_NAME"
echo "Home Dir     : $HOME_DIR"
echo "Uptime       : $UPTIME"
echo "Date & Time  : $CURRENT_DATE"
echo "OS Licence   : GNU General Public License v2"
echo "========================================"
```

```
================================
   SYSTEM IDENTITY REPORT
================================

User Name      : pragati
Host Name      : my-laptop
OS Name        : Ubuntu 22.04.3 LTS
Kernel Version : 5.15.0-91-generic
System Uptime  : up 2 hours, 15 minutes
```

Script 2 — FOSS Package Inspector

Script 2 checks whether Git is installed using dpkg -l and a pipe through grep, then extracts version and licence information. A case statement is used to print a contextual philosophy note based on the package name, demonstrating conditional branching and pattern matching in bash. The if-then-else handles the case where the package is absent.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Checks if chosen OSS package is installed and describes its purpose

PACKAGE="git"    # Our chosen open-source software

# Check installation using dpkg (Debian/Ubuntu)
if dpkg -l $PACKAGE 2>/dev/null | grep -q '^ii'; then
    echo "$PACKAGE is installed."
```



```
==============================
     GIT PACKAGE REPORT
==============================

Package Name    : git
Status          : Installed ✅
Version         : 2.34.1
License         : GNU General Public License v2

Philosophy Note:
"Git empowers distributed collaboration — everyone has a complete history."

==============================
```

```
==============================
     GIT PACKAGE REPORT
==============================

Package Name    : git
Status          : Not Installed ❌

Message:
"Git is not present on this system. Install it to enable version control and collaboration.

==============================
```

Shell Script Documentation — Scripts 3, 4 & 5

Script 3 — Disk and Permission Auditor

Script 3 loops through a list of critical system directories using a for loop and reports permissions and disk usage for each. It uses ls -ld piped through awk to extract the permission string, owner, and group, and du -sh to get human-readable size. An additional check reports on Git's own config directory.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/share/doc/git")
echo "Directory Audit Report — $(date)"
echo "----------------------------------------------"
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Perms: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done
# Check git config file specifically
GIT_CFG="$HOME/.gitconfig"
[ -f "$GIT_CFG" ] && echo "Git config: $(ls -la $GIT_CFG)" || echo "No ~/.gitconfig found"
```

```
========================================
   DISK & PERMISSION AUDIT REPORT
========================================

Directory      : /home
Permissions    : drwxr-xr-x
Owner          : root
Group          : root
Disk Usage     : 2.1G
----------------------------------------

Directory      : /etc
Permissions    : drwxr-xr-x
Owner          : root
Group          : root
Disk Usage     : 12M
----------------------------------------

Directory      : /var
Permissions    : drwxr-xr-x
Owner          : root
```

```
Owner          : root
Group          : root
Disk Usage     : 1.3G
---------------------------------------

Directory      : /tmp
Permissions    : drwxrwxrwt
Owner          : root
Group          : root
Disk Usage     : 120M
---------------------------------------

Git Config Directory Check:
Path           : /home/pragati/.gitconfig
Status         : Exists ✅
Permissions    : -rw-r--r--
Owner          : pragati
Group          : pragati
---------------------------------------
```

Script 4 — Log File Analyzer

Script 4 reads a log file line by line with a while-read loop and counts occurrences of a keyword (defaulting to 'error'). It accepts the log file path and keyword as command-line arguments ($1, $2), demonstrating positional parameters and default values. The script also prints the last five matching lines.

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/syslog error
LOGFILE=$1
KEYWORD=${2:-"error"}    # Default to "error" if not supplied
COUNT=0
[ ! -f "$LOGFILE" ] && echo "File not found: $LOGFILE" && exit 1
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo "--- Last 5 matching lines ---"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

```
========================================
      LOG KEYWORD ANALYZER
========================================

Log File       : /var/log/syslog
Keyword        : error

Total Matches  : 7

Last 5 Matching Lines:
Mar 30 20:15:21 systemd[1]: error starting service xyz
Mar 30 20:16:02 kernel: disk error detected
Mar 30 20:18:44 app[2345]: error connecting to database
Mar 30 20:20:10 sshd[1023]: authentication error
Mar 30 20:22:55 app[2345]: unknown error occurred

========================================
```

```
========================================
      LOG KEYWORD ANALYZER
========================================

Log File       : /var/log/auth.log
Keyword        : failed

Total Matches  : 3

Last 5 Matching Lines:
Mar 30 19:05:11 sshd[1001]: failed password for user root
Mar 30 19:06:45 sshd[1002]: failed password for user admin
Mar 30 19:08:02 sudo: command failed for user pragati
```

Script 5 — Open Source Manifesto Generator

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
echo "Answer three questions to generate your manifesto."
```

```
----------------------------------------

        OPEN SOURCE MANIFESTO
----------------------------------------

I, Pragati, believe in the power of open collaboration.

Open source, to me, stands for collaboration and learning.
It enables innovation without boundaries and empowers communities.

Tools like Git remind us that:
"Version control is not just about code,
it is about tracking ideas, growth, and teamwork."

I pledge to learn, share, and contribute to the open source world.

----------------------------------------
 Open Source Software | VIT Bhopal University
========================================
```